A Verification-First Agentic Harness for Large-Scale Code Migration

Project proposal · Autonomous Software Engineering

1. Problem statement and motivation

Large-scale code migrations (runtime upgrades such as Java 8 to 17, library swaps, or translating C to Rust) cost engineer-years and recur at every company with a large codebase: Airbnb estimated 1.5 engineer-years for one test migration, Slack over 10,000 hours for another. Platform teams carry this work, and reviewers become the bottleneck once edits are automated. AI agents now do much of the editing but still fail often (about 52-56% success on Java upgrades, about 22% test pass on C to Rust), and they cannot show that migrated code behaves like the original. Passing tests can hide behaviour changes, and agents have been seen deleting failing tests to get green builds. Yet in a migration the original program already defines correct behaviour, so equivalence can be checked mechanically. Current pipelines do not exploit this.

2. Existing approaches

- **Deterministic codemods** (OpenRewrite, Comby): reliable on mechanical patterns; the long tail of non-mechanical cases stays manual.
- **Industrial LLM pipelines** (Google, Airbnb, Amazon, Stripe): fixed steps, build-and-test retry loops, human review of every change. In-house, language-specific, and they rely on test quality with no proof of equivalence.
- **General coding agents** (Claude Code, Codex CLI, OpenHands): not migration-aware, can game tests, and results vary from run to run.
- **Verified translation research** (VERT, Heimdall): proves equivalence (VERT raised provably equivalent Rust translations from 1% to 42%), but for one language pair and small programs.

Opportunity: combine an open, language-agnostic agent harness with per-change equivalence checking.

3. Objectives and scope

Our team will build a research prototype that delivers:

- a migration core with pluggable language packs, and two packs: Java 8 to 17 and C to Rust;
- a checking ladder for every changed function: build, original tests, differential testing, then a bounded equivalence proof with counterexample-driven repair;
- integrity rules (no test deletion, no version downgrades, no invented dependencies) and the option to abstain with a diagnosis;
- an evaluation against baseline agents using the same model and budget.

Out of scope: production readiness or a user interface; whole-program proofs; proofs for I/O, concurrency- or reflection-heavy code (these stop at testing); other languages; training or fine-tuning models.

4. Proposed direction

A fixed pipeline with agentic steps inside it: **discover** the sites to change, **plan and order** them leaf-first, **transform** one unit at a time with curated context (the unit, its callers, target-API docs, earlier successful edits), **check**, **repair** from the failure with capped retries, then **land** with a verification record or **abstain**. Checks run cheapest first; each language pack supplies the tools for each level.

Formal verification: the top check level uses bounded model checking (JBMC for Java; Kani or CBMC for Rust) to formally prove that each small, self-contained migrated function is equivalent to its original. Where the tools reach, this replaces "the tests pass" with a machine-checked guarantee; elsewhere the harness falls back to testing and reports which level each change reached.

To prove equivalence, a small wrapper calls the old and new function on the same symbolic input and asserts equal outputs. The checker either proves this for all inputs up to a size bound or returns a concrete counterexample, which becomes the model's next repair prompt. The model never defines what "equivalent" means, and a mutation check confirms each proof is not vacuous.

Why agentic: the hard, long-tail sites need exploration, tool use (build, tests, checker) and repair from feedback, which scripts and one shot LLM calls cannot provide. Agency stays bounded inside fixed steps, since constrained pipelines have outperformed free roaming agents in published migration work.

5. Success and quality attributes

Evaluated on subsets of FreshBrew (Java) and CRUST-bench (C to Rust), 3 runs per task, against mini-SWE-agent, a CLI agent and OpenRewrite, with the same model and budget.

- **Correctness:** benchmark success; at least 10 points above the best LLM baseline on one benchmark.
- **Verification strength:** share of changed functions reaching each check level; behaviour changes found in test-passing migrations.
- **Proof trustworthiness:** at least 90% of deliberately planted bugs caught by accepted proofs.
- **Integrity:** zero deleted or weakened tests, downgrades or invented dependencies.
- **Reliability:** mean success over 3 runs, its spread, and the share of tasks solved in all 3 runs.
- **Resource efficiency:** tokens, cost and time per successful migration, no higher than the CLI-agent baseline.